

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. RISC-V RV32I Architecture
6. Overall Processor Architecture
7. Detailed Module-Level Design
8. Instruction Execution and Data Flow
9. Control Signal Generation
10. Clocking Strategy and Timing
11. Memory Organization and Address Mapping
12. Supported Instruction Set
13. Critical Path and Performance Analysis
14. Verification and Testbench
15. Simulation Results
16. FPGA Implementation
17. Design Challenges and Solutions
18. Limitations
19. Future Scope
20. Conclusion

1. Abstract

This project presents the design and FPGA implementation of a 32-bit RISC-V processor based on the RV32I instruction-set architecture and a single-cycle datapath organization. The processor is structured around the conventional instruction fetch, instruction decode, execute, memory access, and write-back functions, with these architectural stages completed within one clock period. The RTL design includes a program counter, instruction memory, register file, immediate-extension logic, control logic, ALU, data memory, branch/jump address generation, and multiplexers for operand, result, and next-PC selection.

The implementation supports arithmetic, logical, shift, comparison, immediate, load/store, conditional branch, and JALR operations documented in the supplied reference material. The design uses separate instruction and data memories, each organized as 256 words of 32 bits, and employs word-aligned indexing using address bits [9:2]. The register file provides two source-register reads and one destination-register write, with register x0 protected from modification.

Verification is performed using simulation-oriented testbench techniques that preload instructions, generate a 10 ns clock, assert reset, and monitor processor signals such as the program counter, instruction, ALU activity, and register state. The accompanying Vivado project metadata identifies Vivado 2024.1 and the Xilinx Artix-7 device xc7a100tcs324-1 as the project target. The report emphasizes not only functional behavior but also the reasoning behind important RTL decisions, including negative-edge state updates, zero-register enforcement, word-aligned memory indexing, control-signal generation, and critical-path limitations.

2. Introduction

A processor can be viewed as a coordinated datapath and control system that repeatedly fetches instructions, interprets them, performs the required operation, optionally accesses memory, and stores the result. Designing such a processor at RTL level provides a direct connection between instruction-set architecture, digital logic design, timing, and FPGA implementation.

RISC-V is an open instruction-set architecture that defines the software-visible behavior of a processor while allowing many different microarchitectural implementations. This project focuses on a 32-bit implementation corresponding to the RV32I base integer model as described in the reference material. Rather than using a commercially available processor IP, the project constructs the principal datapath and control functions in Verilog HDL.

The selected single-cycle organization is particularly useful for academic study because the dataflow of an individual instruction can be traced directly across the processor. The trade-off is that the clock period must accommodate the longest combinational instruction path. Consequently, the architecture favors conceptual simplicity and straightforward verification over the performance advantages of pipelining.

Why was a single-cycle architecture selected?

The single-cycle organization allows every supported instruction to complete its architectural work within one clock period. This makes the relationship between the instruction encoding, control signals, datapath, and write-back operation easy to observe and verify. The main performance drawback is the long critical path, discussed later in the report.

3. Problem Statement

The engineering problem addressed in this project is to design and implement a functional 32-bit RISC-V processor in Verilog HDL, with a clearly defined datapath and control path, supporting a representative RV32I instruction subset and capable of simulation and FPGA implementation. The design must correctly transfer an instruction from memory through decoding and execution, perform data-memory operations when required, select the appropriate write-back result, and determine the next program-counter value for sequential, branch, and jump control flow.

Formal problem statement

Design, implement, simulate, and validate a 32-bit RISC-V RV32I single-cycle processor using Verilog HDL, integrating instruction fetch, decode, execution, memory access, write-back, control generation, and next-PC selection in an FPGA-oriented RTL design.

4. Objectives

- Design a 32-bit RISC-V single-cycle datapath using modular Verilog RTL blocks.
- Implement the program counter, instruction memory, register file, immediate-extension logic, control unit, ALU, data memory, multiplexers, and branch/jump address generation.
- Support the documented R-type, I-type, load, store, branch, and JALR instruction categories.
- Generate and route control signals so that each instruction activates only the required datapath functions.
- Verify sequential execution, arithmetic/logic behavior, memory operations, and branch behavior using simulation.
- Understand implementation-level timing decisions, especially the use of negative-edge updates and the resulting critical-path constraint.
- Target the design to a Xilinx FPGA using Vivado and establish a foundation for future pipelined and higher-performance implementations.

5. RISC-V RV32I Architecture

The processor operates on 32-bit data and uses 32 integer registers addressed by 5-bit register indices. The instruction formats and fields determine which source operands are read, which destination register is written, which immediate is constructed, and which ALU or control-flow operation is performed.

5.1 Register Set and the x0 Register

The register file contains registers x0 through x31. Register x0 is architecturally defined as the zero register. Reads from x0 return zero, while a write operation targeting register index 0 is suppressed by the RTL. In the documented implementation, registers are initialized to their index values for debugging and simulation visibility; this initialization is a verification convenience rather than a requirement of the RISC-V architecture.

Design Question: Why is register x0 hardwired to zero?

A constant-zero register simplifies instruction sequences and hardware control. It can act as a zero operand without dedicating a separate immediate-generation path, and it also enables simple forms of moves, comparisons, and address generation. In RTL, suppressing writes to rd=0 preserves the architectural invariant that x0 remains zero.

5.2 Instruction Fields

The 32-bit instruction contains fields such as opcode, rd, rs1, rs2, funct3, and, for relevant instructions, funct7 and immediate fragments. The opcode identifies the broad instruction category, while funct3 and funct7 refine the specific operation for instruction classes that share the same opcode.

5.3 Immediate Formats

The reference design explicitly constructs I-type, S-type, and B-type immediates and sign-extends them to 32 bits. For example, an I-type immediate uses instruction bits [31:20], an S-type immediate combines [31:25] and [11:7], and a B-type branch displacement rearranges instruction bits and appends a zero least-significant bit because branch targets are aligned.

Why is sign extension required?

The immediate field is narrower than the 32-bit datapath. Sign extension preserves the numerical meaning of a signed two's-complement immediate when it is used as a 32-bit operand. Without sign extension, negative offsets and negative immediate arithmetic would produce incorrect results.

Format	Purpose	Immediate construction	Typical instructions
R-type	Register-register operation	No immediate	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
I-type	Immediate ALU or load/jump operation	inst[31:20] sign-extended	ADDI, ANDI, SLLI, LW, JALR
S-type	Store operation	{inst[31:25], inst[11:7]} sign-extended	SW
B-type	Conditional branch	Rearranged branch displacement, LSB=0	BEQ, BNE, BLT, BGE, BLTU, BGEU

6. Overall Processor Architecture

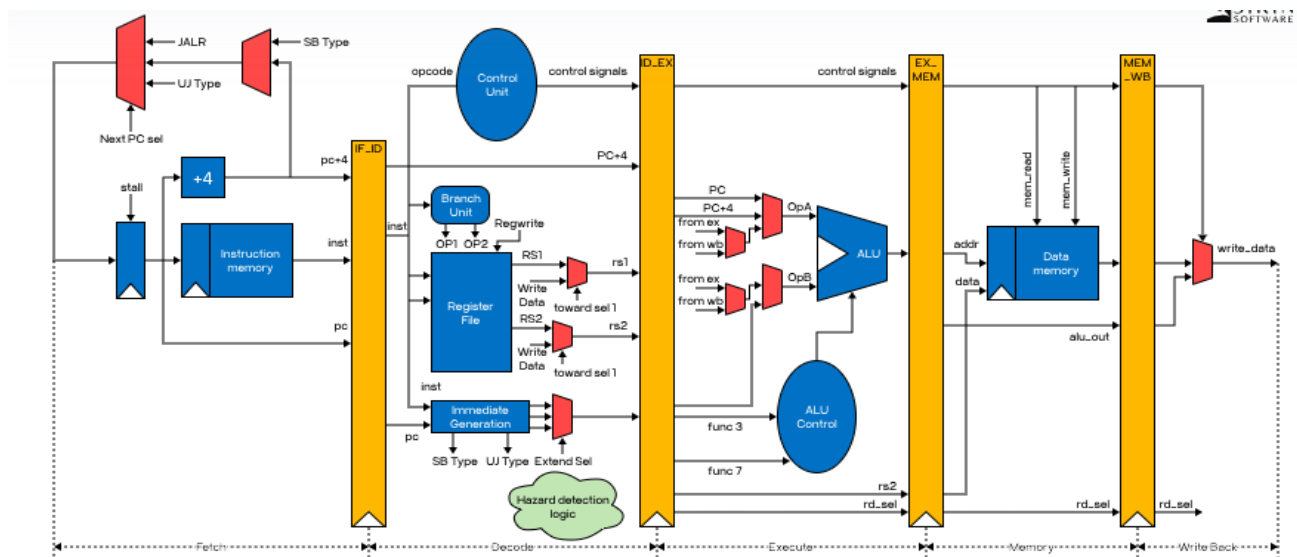
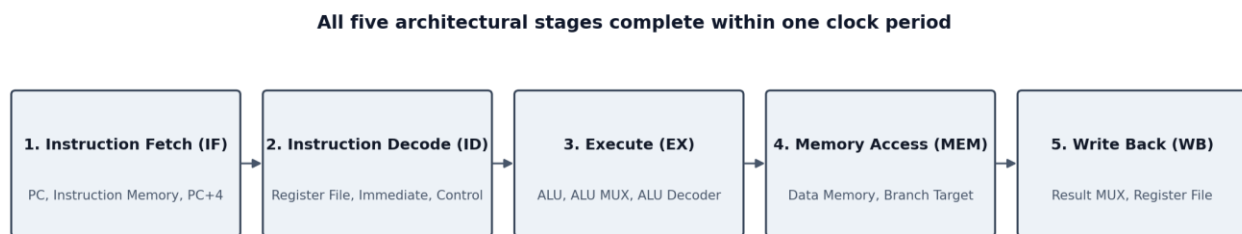


Figure 1. Functional block diagram of the RISC-V single-cycle processor.

The processor is organized as a datapath controlled by decoded instruction information. The PC register supplies the address for instruction memory. The fetched instruction is decoded into register indices, immediate information, and control signals. The register file supplies source operands, while the immediate-extension unit provides a 32-bit immediate when required. A source multiplexer selects the second ALU operand. The ALU performs the selected operation and produces either an arithmetic/logic result or an address/condition value used by later logic.

For load and store instructions, the ALU result becomes the effective memory address. The data memory handles the corresponding read or write. For branches and jumps, separate next-PC logic determines whether execution continues at PC+4 or transfers to a computed target. Finally, the result multiplexer selects the value that is written back to the destination register.

6.1 Architectural Stage Flow



The RTL implementation uses different clock edges for selected state updates; see Section 10.

Figure 2. Architectural five-stage instruction flow compressed into one clock period.

The processor is described using the familiar IF, ID, EX, MEM, and WB terminology even though it is not pipelined. These are functional stages of one instruction, not five simultaneously executing pipeline stages. The complete work of the selected instruction is resolved within one clock period, subject to the implementation timing of the combinational paths.

6.2 Data Flow

The primary dataflow is from the current PC to instruction memory, from the decoded instruction to the register file and immediate generator, through the ALU, and then either to data memory or to the write-back multiplexer. The selected result returns to the register file. In parallel, PC+4 and branch/jump target candidates are evaluated so the next-PC multiplexer can select the correct control-flow destination.

6.3 Control Flow

The control unit interprets the opcode and instruction fields and generates signals that determine operand selection, ALU behavior, memory access, register write enable, result selection, immediate format, and next-PC source. The controller therefore coordinates the datapath rather than performing the arithmetic itself.

Design Question: What is the difference between datapath and control path?

The datapath carries and transforms operand values: registers, immediate values, ALU inputs/outputs, memory data, and PC values. The control path determines which datapath route is active and which operation is performed. For example, ALUSrc does not carry data; it tells the operand multiplexer which data source to select.

7. Detailed Module-Level Design

7.1 Program Counter Register

Stores the current instruction address and updates the architectural PC using the next-PC value selected by the PC multiplexer. The reference implementation updates the PC on the negative clock edge.

Design Question: Why is a separate PC register required?

The current instruction address must persist as state between clock events. Combinational logic alone cannot retain the address of the current instruction.

7.2 Instruction Memory

Stores 32-bit instructions in a 256-word array. The PC is converted into a memory index using bits [9:2], providing word-aligned instruction access. The reference design describes instruction-memory read as combinational.

Design Question: Why is instruction memory separate from data memory?

The separation is a simple Harvard-style organization that allows instruction fetch and data storage to be modeled independently.

7.3 PC+4 Adder

Adds 4 to the current PC for normal sequential execution. Because instructions are 32 bits = 4 bytes, a correctly aligned sequential instruction address advances by four bytes.

Design Question: Why is the PC incremented by 4 rather than 1?

The design works with 32-bit instructions, which occupy four byte addresses. Therefore the next naturally aligned instruction is at PC+4.

7.4 Register File

Implements 32 general-purpose registers with two source reads and one destination write. Source registers are selected by rs1 and rs2, while rd selects the write-back destination. Writes to rd=0 are blocked to preserve x0=0.

Design Question: Why are two read ports needed?

R-type and branch instructions commonly require two source operands, rs1 and rs2, in the same instruction execution. Two read paths expose both values without an extra sequential cycle.

7.5 Immediate Extension Unit

Extracts the immediate fragments required by the current instruction format and sign-extends them to 32 bits. B-type immediates are rearranged for branch displacement generation.

7.6 Controller

Generates the main control signals from the instruction opcode and related control fields. It determines whether the current operation writes a register, accesses memory, uses an immediate, writes back memory data, or changes the PC.

7.7 ALU Decoder

Maps instruction-class information and function fields to the internal ALU control code. The documented ALU control mapping includes ADD=0, SUB=1, SLL=2, SLT=3, SLTU=4, XOR=5, SRL=6, SRA=7, OR=8, and AND=9.

7.8 ALU Source MUX

Selects the second operand of the ALU. The documented sources are rs2, the full immediate, or a 5-bit shift amount. This prevents separate ALUs from being required for register and immediate instructions.

Design Question: Why does the ALU need an imm5 option for shifts?

Shift-immediate instructions encode the shift amount in a 5-bit field. Selecting the narrower shift amount directly avoids treating it as an ordinary full arithmetic immediate.

7.9 ALU

Performs the arithmetic, logical, shift, and comparison operations required by the supported instruction subset. For load/store operations it also computes effective addresses.

7.10 Data Memory

Implements a 256-word, 32-bit data array. The reference design uses synchronous writes on the positive clock edge and a combinational read path for load operations.

Design Question: Why is data-memory write synchronous while instruction-memory read is combinational?

The reference architecture models instruction lookup as a direct combinational array read, while store operations are committed on a clock edge. This cleanly separates continuous observation of memory contents from state-changing writes.

7.11 Branch Target Adder

Combines the current PC with the sign-extended B-type branch displacement to create the branch target candidate.

7.12 Result MUX

Selects the value that is finally written to the destination register. The documented sources are the ALU result, memory read data, or PC+4.

Design Question: Why are three write-back sources required?

R-type and I-type ALU instructions write the ALU result, LW writes memory data, and JALR writes PC+4 as the link value.

7.13 PC MUX

Selects the next program counter from sequential PC+4, a branch target, or the JALR target derived from the ALU path.

8. Instruction Execution and Data Flow

The most useful way to understand the processor is to trace one instruction from its current PC to its architectural result. The following subsections describe representative instruction classes and identify the data and control signals involved.

8.1 R-Type Example: ADD x3, x1, x2

- The PC addresses instruction memory and the 32-bit ADD instruction is fetched.
- The decoder identifies opcode 0110011 and extracts $rs1=x1$, $rs2=x2$, and $rd=x3$.
- The controller selects a register-register ALU operation, enables register write-back, and selects PC+4 as the next PC.
- The register file outputs the current contents of x1 and x2.
- The ALU source MUX selects rs2 as the second ALU operand.
- The ALU decoder selects ADD and the ALU computes $x1+x2$.
- The result MUX selects the ALU result and the value is written to x3 at the designated register-write edge.
- The PC logic selects PC+4 for the next sequential instruction.

Question to expect: Where does the destination register come from?

The rd field is instruction bits [11:7]. The decoder passes this 5-bit index to the register-file write port.

8.2 I-Type Example: ADDI x3, x1, 10

- Fetch identifies an opcode of 0010011 for an immediate ALU instruction.
- The immediate generator extracts bits [31:20] and sign-extends the 12-bit value to 32 bits.
- The register file reads $rs1=x1$.
- The ALU source MUX selects the immediate rather than rs2.
- The ALU decoder selects ADD and the ALU computes $x1+10$.
- The ALU result is selected by the result MUX and written to x3.
- The next PC is PC+4.

8.3 Load Example: LW x11, 0(x0)

- At the current PC, instruction memory supplies the LW instruction.
- The decoder identifies opcode 0000011, rs1=x0, rd=x11, and an immediate of 0.
- The controller enables register write and memory read, selects the immediate as the ALU input source, and chooses memory data for write-back.
- The ALU computes the effective address $x0 + 0 = 0$.
- The data memory is read at the computed word address.
- The result MUX selects memory data.
- The loaded 32-bit word is written to x11.

Why is LW central to timing analysis?

LW exercises the fetch, decode, register-read, immediate, ALU address-generation, memory-read, and write-back datapath in sequence. Therefore it is a natural representative of the longest instruction path in a single-cycle design.

8.4 Store Example: SW x1, 0(x0)

- The instruction decoder identifies a store opcode of 0100011 and constructs an S-type immediate.
- The register file supplies rs1 and rs2; rs2 contains the value to be stored.
- The ALU source MUX selects the sign-extended store offset.
- The ALU computes the effective address $rs1 + \text{immediate}$.
- The data memory receives rs2 as write data and commits the write when MemWrite is active at the designated clock edge.
- No register write-back occurs for SW; the PC advances to PC+4.

8.5 Branch Example: BEQ

- The decoder identifies opcode 1100011 and constructs the B-type immediate.
- The register file supplies rs1 and rs2.
- The ALU compares the operands according to the branch operation. For BEQ the branch condition is satisfied when the operands are equal.
- The branch target adder forms PC plus the sign-extended branch displacement.
- The PC multiplexer selects the branch target when the branch condition is true; otherwise it selects PC+4.
- No destination register is written for the branch instruction.

8.6 JALR Example

JALR uses an I-type immediate and an ALU-derived target address. The result MUX selects PC+4 for write-back so the destination register can hold the link address, while the PC MUX selects the jump target. This creates a deliberate separation between the value written to rd and the address used to fetch the next instruction.

Question to expect: Why are write-back and next-PC selection separate?

A control-flow instruction may need to write one value to a register while sending a different value to the PC. JALR is the clearest example: rd receives the link address PC+4, while the PC receives the computed jump target.

9. Control Signal Generation

The processor uses a compact set of control signals to coordinate the datapath. The reference material identifies PCSrc, ALUSrc, ResultSrc, and ImmSrc as multi-bit selectors, together with register-write and memory-control signals.

Signal	Purpose	Documented meaning
PCSrc[1:0]	Selects the next PC source	00 = PC+4; 01 = branch target; 10 = jump/JALR target
ALUSrc[1:0]	Selects ALU second operand	00 = rs2; 01 = immediate; 10 = imm5/shift amount
ResultSrc[1:0]	Selects register write-back data	00 = ALU result; 01 = memory data; 10 = PC+4
ImmSrc[1:0]	Selects immediate format	00 = I-type; 01 = S-type; 10 = B-type
RegWr	Enables register-file write	1 for instructions that write rd
MemWrite	Enables data-memory write	1 for SW
MemRead	Enables/identifies data-memory read	1 for LW

9.1 Instruction-to-Control Relationship

Class	Opcode	RegWr	ALUSrc	ResultSrc	ImmSrc	PCSrc	Memory / note
R-type	0110011	1	00	00	00	00	No memory access
I-type ALU	0010011	1	01 or 10	00	00	00	No memory access
LW	0000011	1	01	01	00	00	MemRead=1
SW	0100011	0	01	X	01	00	MemWrite=1
Branch	1100011	0	00	X	10	01 if taken	Conditional PC select
JALR	1100111	1	01	10	00	10	ALU target for PC

Design Question: Why does the controller not directly perform the ALU operation?

The controller establishes the instruction class and high-level control. The ALU decoder performs the finer operation mapping using function fields. This separation keeps the control logic modular and allows several instruction classes to share the same ALU datapath.

10. Clocking Strategy and Timing

The supplied reference implementation uses more than one clock edge for state-changing elements: the PC register and register-file write logic update on the negative edge, while the data-memory write operation is associated with the positive edge. This is an implementation-level timing strategy rather than a requirement imposed by the RISC-V ISA.

10.1 Why is negedge used for the PC?

Using the negative edge as the update event means the new PC becomes available after that edge and the combinational instruction path has the following half-cycle interval to propagate through instruction memory and decode logic. The choice also separates state updates from other positive-edge events in the described testbench-oriented implementation.

10.2 Why is negedge used for register-file writes?

A register value produced by the combinational datapath can be allowed to settle before the negative edge, at which time the write-back value is committed. This can reduce read/write ordering ambiguity in simulation. It should be understood as a chosen RTL timing convention, not as a universal rule that every single-cycle processor must use negedge.

10.3 Why does data memory use posedge?

The documented data-memory model commits stores synchronously on the positive edge. This makes the memory write an explicit state transition. Loads, by contrast, are described as combinational reads from the memory array, so the read data can propagate toward the write-back logic without waiting for another storage edge.

Design Question: What would happen if every state element used the same edge?

A same-edge implementation can still be correct, but the RTL and testbench must be designed so that combinational paths, setup requirements, and simulation scheduling are unambiguous. The supplied implementation instead distributes selected updates across clock edges, which simplifies the intended sequencing in this model.

10.4 Combinational vs Sequential Logic

Registers such as the PC and register-file storage hold state and therefore use edge-triggered sequential logic. The ALU, multiplexers, immediate-generation logic, instruction decoding, and documented instruction-memory read operate as combinational functions. Data-memory write is sequential, while data-memory read is combinational in the documented model.

11. Memory Organization and Address Mapping

Both instruction and data memory are described as 256-word arrays of 32-bit entries. The memory index uses address bits [9:2], which produces eight index bits and therefore $2^8 = 256$ possible locations. With 32 bits per word, each memory contains $256 \times 4 = 1024$ bytes, or 1 KB.

11.1 Why are bits [1:0] ignored?

A 32-bit word contains four bytes. In a naturally word-aligned address, the two least significant address bits identify the byte position inside the four-byte word and are therefore 00. Because this design models word access only, the two lower bits are not required for selecting the array entry. Dropping them converts a byte address into a word index.

11.2 Address Examples

- Address 0x000 maps to mem[0].
- Address 0x004 maps to mem[1].

- Address 0x008 maps to mem[2].
- Address 0x00C maps to mem[3].
- Address 0x010 maps to mem[4].

11.3 What Happens to an Unaligned Address?

Because the array index is formed only from bits [9:2], an address such as 0x005 produces the same index as 0x004. In the documented simplified model, the lower bits are silently ignored and no alignment exception is generated. A full production processor would normally implement the architectural rules for alignment, exceptions, and byte/halfword accesses rather than silently aliasing the address.

Design Question: Why not use the complete 32-bit address as the array index?

The array has only 256 entries. Using all 32 bits would imply an impractically large direct array index space and would not match the implemented memory capacity. Address slicing is the hardware mapping from the processor address space to the implemented storage array.

12. Supported Instruction Set

The documented processor supports a representative subset spanning arithmetic/logic, immediate operations, load/store, conditional branches, and JALR. The instruction list below follows the supplied reference document.

Instruction class	Opcode	Supported operations
R-type	0110011	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
I-type ALU	0010011	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
Load	0000011	LW
Store	0100011	SW
Branch	1100011	BEQ, BNE, BLT, BGE, BLTU, BGEU
JALR	1100111	JALR

12.1 ALU Control Encoding

ALU control code	Operation
4'h0	ADD
4'h1	SUB
4'h2	SLL
4'h3	SLT
4'h4	SLTU
4'h5	XOR
4'h6	SRL

ALU control code	Operation
4'h7	SRA
4'h8	OR
4'h9	AND

Design Question: Why are signed and unsigned comparisons separate?

SLT and SLTU interpret the same bit patterns under different numerical rules. The signed comparison treats the operands as two's-complement values, while the unsigned comparison treats them as non-negative binary magnitudes. The distinction is therefore architectural, even though both operations use the same ALU hardware conceptually.

13. Critical Path and Performance Analysis

In a single-cycle processor, the clock period must be long enough for the slowest instruction to complete its required combinational path and satisfy the setup requirements of its destination state element. The critical path is therefore an architectural consequence of requiring all work for an instruction to fit within one clock period.

13.1 Representative Critical Path

The load-word instruction is the natural representative of the longest path because it uses instruction fetch, instruction decoding and register read, immediate generation, ALU address calculation, data-memory read, result selection, and register-file write-back.

A more precise timing view is: PC/register state -> instruction memory -> decode and control -> register-file read / immediate generation -> ALU -> data memory -> result MUX -> register-file input, followed by the destination register setup requirement. The reference document summarizes this as IF -> ID -> EX -> MEM -> WB.

Design Question: Why does LW usually determine the critical path?

R-type instructions normally stop after the ALU and result MUX, while stores do not require register write-back. LW requires the additional data-memory access and then must carry the loaded value through the write-back selection before it reaches the destination register. Therefore LW exercises more serial combinational logic.

13.2 Clock Frequency Relationship

For an idealized timing model, the maximum clock frequency is approximately the inverse of the minimum safe clock period. In practice, the clock period must include logic delay, routing delay, setup time, clock uncertainty, and any other implementation-specific margins. Therefore a numerical frequency obtained from a simple sum of assumed gate delays should not be reported as the measured FPGA maximum frequency.

13.3 Why Single-Cycle Performance is Limited

- The clock period must accommodate the longest instruction, even when most instructions need less logic.
- No instruction can overlap its execution with another instruction because the architecture is not pipelined.
- Long combinational paths increase timing pressure and reduce the maximum achievable clock rate.
- Any slow memory model or long routing path directly increases the cycle-time requirement.

13.4 How Pipelining Changes the Problem

A pipelined design inserts state registers between IF, ID, EX, MEM, and WB so different instructions can occupy different stages simultaneously. The clock period is then determined primarily by the slowest stage rather than the sum of all five stages. The trade-off is added pipeline control complexity, including data hazards, forwarding, stalls, and control hazards.

14. Verification and Testbench

Verification checks whether the RTL implementation performs the expected architectural operations for representative instructions and control-flow cases. The reference testbench strategy includes clock generation, reset handling, instruction preloading, and real-time monitoring.

14.1 Testbench Strategy

1. Generate a periodic clock with a 10 ns period, corresponding to 100 MHz in simulation.
2. Apply reset so the PC and documented initialization behavior start from known values.
3. Preload instruction memory with test instructions.
4. Run the processor for enough clock events to observe sequential execution and branch behavior.
5. Monitor PC, instruction, ALU-related signals, register values, and memory state to determine whether the expected operation occurred.

14.2 Representative Test Cases

Test instruction	Verification purpose
SW x1, 0(x0)	Tests effective-address generation and data-memory write.
LW x11, 0(x0)	Tests memory read and memory-to-register write-back.
BEQ x1, x1, +8	Tests a condition that should be true.
BNE x1, x2, +2	Tests a not-equal branch condition.
BLT x11, x2, +2	Tests signed less-than branching.
BGE x2, x1, +8	Tests signed greater-or-equal branching.

14.3 Monitoring and Debugging

The reference testbench monitors internal signals such as PC, instruction, ALU output, and register values. Instruction fields can also be extracted directly from the DUT instruction bus, for example rs1 from bits [19:15], rs2 from [24:20], and rd from [11:7]. This makes it possible to relate each waveform transition to the instruction being executed.

Design Question: How do you know the processor executed the instruction correctly?

Functional correctness should be established by checking both intermediate signals and final architectural state. For example, an LW test is stronger when the waveform shows the decoded rs1/rd fields, the calculated memory address, the returned memory data, and the subsequent register-file write rather than checking only the final register value.

15. Simulation Results

The simulation and implementation results were obtained from the Vivado project using a 10 ns clock period. The captured evidence includes the elaborated RTL schematic, the simulation waveform, and the simulator console log. Together, these views provide visibility into module connectivity, program-counter progression, ALU results, register updates, memory activity, and the execution sequence of the programmed instruction set.

15.1 Observed Sequential Execution

The waveform shows the PC progressing through word-aligned instruction addresses in 4-byte increments during normal sequential execution. The instruction memory uses PC[9:2] as the array index, so successive instruction words are selected as the PC advances.

15.2 Observed Load/Store Behavior

The captured simulation shows the load/store portion of the program operating through the ALU-generated effective address and the data-memory interface. The corresponding register value is then observed at the register-file write-back output for the load operation.

15.3 Observed Control-Flow Behavior

The waveform captures changes in the next-PC path when control-flow instructions are executed. The implemented PC MUX selects among PC+4, the branch target, and the ALU-derived JALR target according to the decoded control signals.

15.4 Simulation and Implementation Evidence

The following figures document the implemented processor as observed in Vivado. The RTL schematic provides a structural view of the connected modules, while the waveform and console log provide dynamic simulation evidence for instruction execution and state changes.

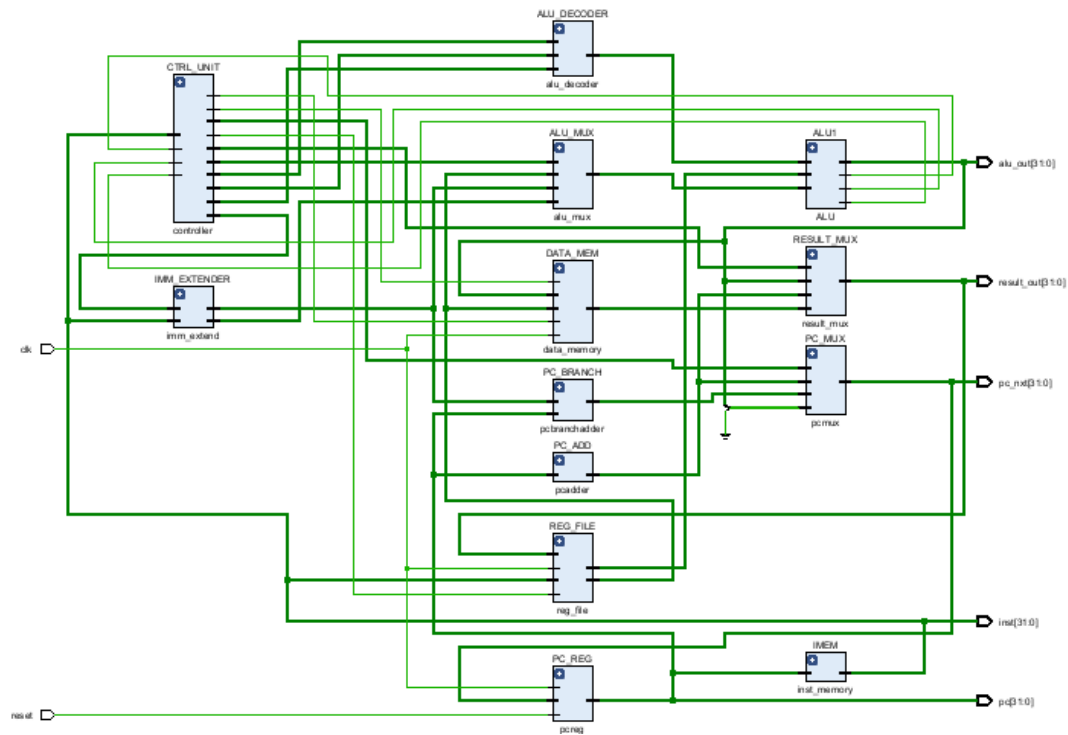


Figure 4. Vivado RTL schematic of the RISC-V single-cycle processor.

The Vivado RTL schematic shows the processor modules connected through the datapath and control signals, including the PC register, instruction memory, register file, immediate extension, ALU datapath, data memory, result MUX, PC MUX, ALU decoder, and control unit.

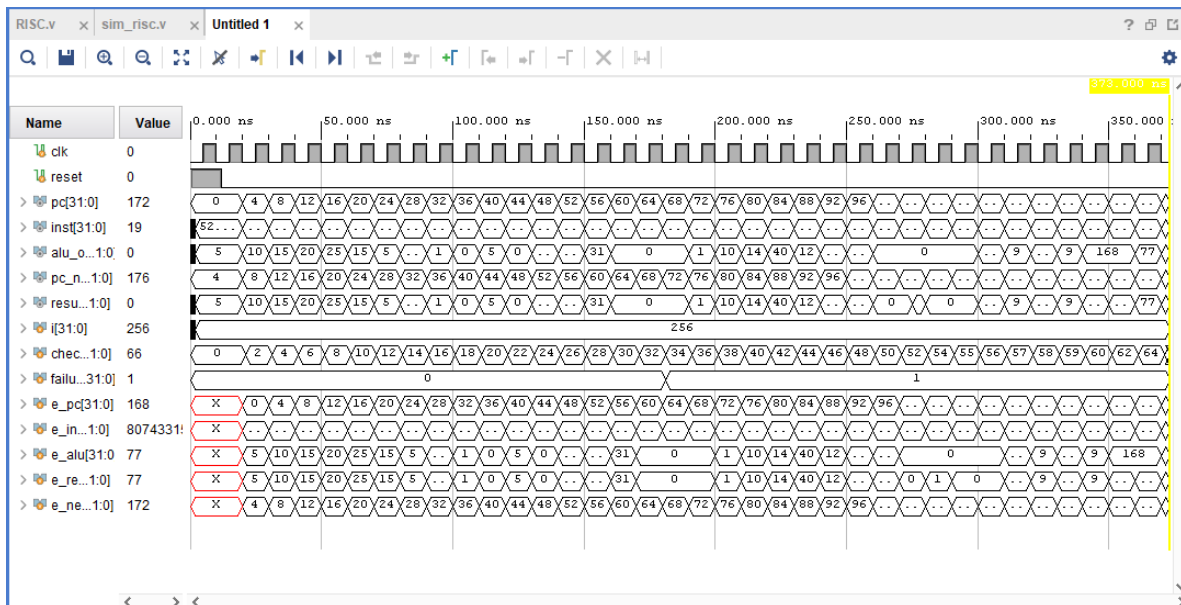


Figure 5. Vivado simulation waveform showing PC, instruction, ALU output, next-PC, result, register values, and testbench status signals.

The waveform provides cycle-level visibility into the processor state. It shows the program counter advancing through the programmed instruction sequence and captures corresponding ALU, result, next-PC, and register activity. The later portion of the trace includes the jump target and target-instruction execution.

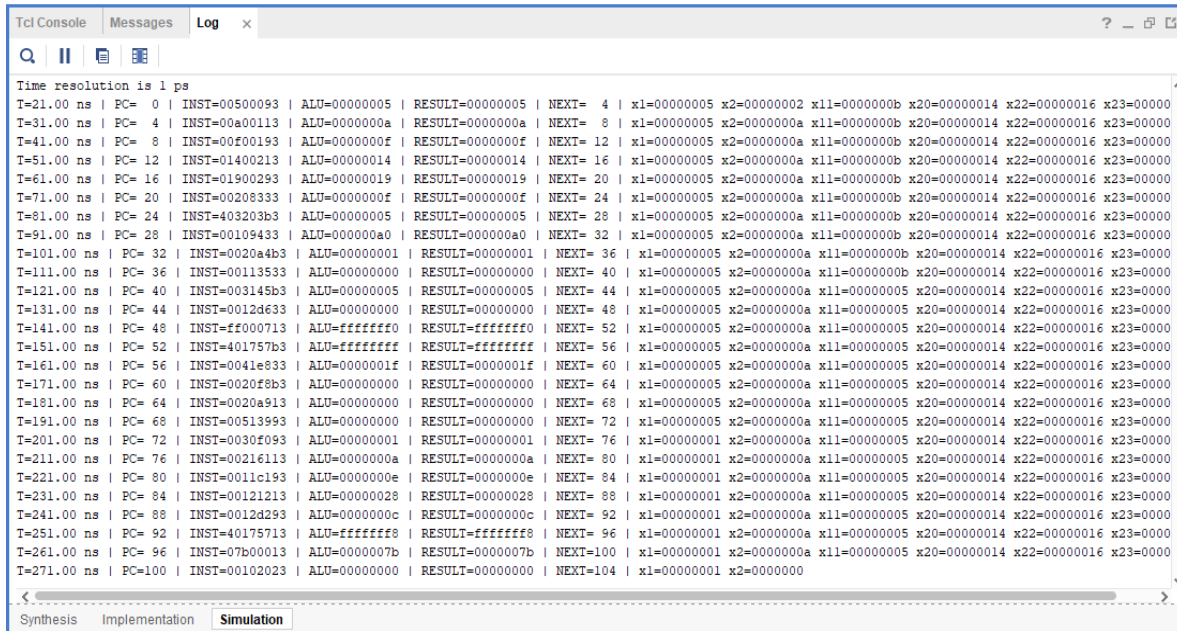


Figure 6. Vivado simulator console log showing cycle-by-cycle execution values.

The console log complements the waveform by reporting the current PC, instruction word, ALU value, write-back result, next-PC value, and selected register contents at each observed execution point. The captured portion demonstrates sequential execution through the arithmetic, logical, shift, and memory portions of the programmed sequence.

16. FPGA Implementation

The supplied Vivado project metadata identifies Vivado 2024.1 and the Xilinx device xc7a100tcs324-1. The project is configured with synthesis and implementation runs and uses a Verilog RTL source set containing modules such as ALU.v, ins_fetch.v, instruction_decode.v, control_unit.v, controller.v, datapath.v, instruction_memory.v, risc_v_top.v, and data memory.v.

16.1 RTL-to-FPGA Flow

1. Create and organize the RTL source modules.
2. Define the design top-level and required constraints.
3. Run RTL elaboration to verify module connectivity and inferred logic.
4. Run synthesis to map the RTL into FPGA logic resources.
5. Run implementation to perform optimization, placement, and routing.
6. Analyze timing and utilization reports.
7. Generate the bitstream and program the target FPGA when hardware validation is required.

16.2 Implementation Metrics

The project metadata establishes the target device and Vivado version, but the supplied XPR file does not include the complete post-implementation resource-utilization and timing numbers required for a quantitative FPGA results table. Those values should be taken directly from the Vivado synthesis and implementation reports before they are included as measured results in a final submission.

Parameter	Documented value
Target tool	Xilinx Vivado 2024.1
Target device	xc7a100tcsg324-1
Design style	Verilog RTL, single-cycle datapath
Memory organization	Separate instruction and data memories
Simulation clock in reference	10 ns period / 100 MHz

17. Design Challenges and Solutions

Challenge	Design issue	Approach used
Instruction-format handling	Different instruction classes place immediate bits in different positions.	Use a dedicated immediate-extension unit selected by ImmSrc.
Shared ALU datapath	Register, immediate, address, and shift operations require different second operands.	Use ALUSrc-controlled multiplexing before the ALU.
Control-flow selection	Sequential, branch, and JALR instructions require different next-PC sources.	Use branch/jump target generation and a PC-source multiplexer.
x0 preservation	The zero register must never change value.	Suppress writes whenever rd=0.
Memory capacity and addressing	The implemented memory is much smaller than the 32-bit architectural address space.	Use the documented address-bit slice [9:2] as the array index.
Clock-edge coordination	Multiple state elements must update predictably without ambiguous simulation ordering.	Use the documented edge-specific update strategy and verify with a consistent testbench.
Single-cycle timing	The slowest instruction determines the entire clock period.	Identify the critical path and treat pipelining as the principal future performance improvement.

18. Limitations

- The processor is single-cycle, so the clock period is constrained by the longest instruction path.
- The documented instruction subset does not include the full RV32I base set; for example, the reference lists JALR but not JAL, LUI, or AUIPC among the supported operations.
- Memory is limited to 256 32-bit words for both instruction and data storage.
- The simplified memory model is word-oriented and does not implement byte or halfword access behavior.
- No pipeline registers, forwarding unit, hazard detector, branch predictor, or cache hierarchy are included in the documented architecture.
- The supplied project metadata does not contain complete post-implementation timing and utilization reports, so quantitative FPGA performance claims must be obtained separately.

Design Question: What is the most important limitation of a single-cycle CPU?

It must choose a clock period long enough for the slowest supported instruction. This means fast instructions do not automatically benefit from their shorter logic paths; they still execute under the same global clock period.

19. Future Scope

1. Convert the single-cycle datapath into a five-stage pipeline with IF/ID, ID/EX, EX/MEM, and MEM/WB registers.
2. Add data forwarding and hazard detection to support dependent instructions without incorrect operand values.
3. Add branch-hazard handling, including stalls, flushing, or branch prediction.
4. Expand instruction support toward a more complete RV32I implementation, including additional immediate, upper-immediate, and jump instructions.
5. Extend the memory subsystem to byte-addressable operation and larger storage capacity.
6. Introduce instruction and data caches to evaluate memory hierarchy effects.
7. Use FPGA timing reports to guide critical-path optimization of the control, ALU, and memory interfaces.
8. Integrate the processor as part of a larger FPGA-based SoC with peripherals and a structured bus interface.

20. Conclusion

The project demonstrates the complete RTL-level construction of a 32-bit RISC-V single-cycle processor using Verilog HDL. The design brings together instruction storage, register storage, immediate generation, control decoding, ALU execution, data memory, and next-PC logic into a coherent datapath. The use of modular blocks makes the processor easier to understand, verify, and extend.

The most important architectural insight is that the five familiar processor functions - instruction fetch, decode, execute, memory access, and write-back - occur as one instruction flow within a single clock period rather than as independent pipeline stages. This provides a clean educational view of processor operation, while the resulting long critical path explains why pipelining is the natural next step for performance improvement.

The project also highlights practical RTL design considerations that are frequently discussed in technical reviews: preserving x0 as a constant zero, choosing clock-edge conventions, interpreting instruction fields, generating sign-extended immediates, mapping byte addresses onto word arrays, and verifying both intermediate signals and final architectural state. These considerations form a strong foundation for progressing from a basic single-cycle CPU toward pipelined and higher-performance RISC-V microarchitectures.